

TS → Process → JS.

Type Script

- Typechecker, Consistency.
- powerful, statically typed Superset of Javascript
- helps write Safer, Robust & Maintainable Code.

Static typing. Type datatype is declared @ time of Programming.

~~Static~~ Dynamic
s = "foo"
n = s.length()

Static ~~Dynamic~~
String S = "foo";
int n = s.length().

Setup.

globally

~~npm~~ npm i -g typescript

tsc filename.ts

↳ This will generate corresponding JS file.

tsc --init

↳ to generate Config file.

(tsconfig.json)

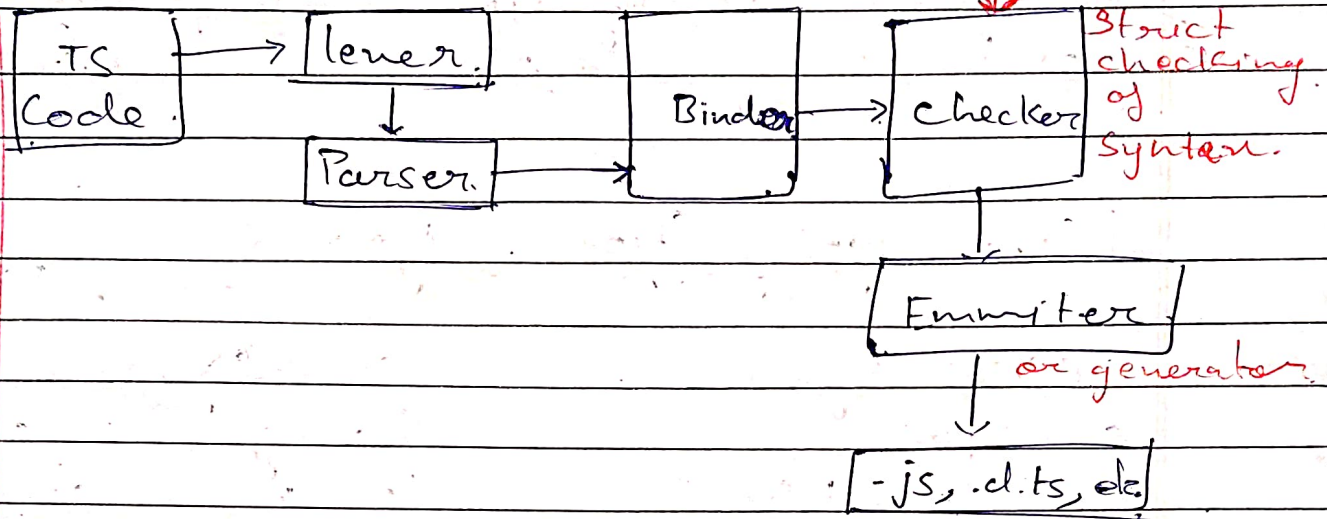
Types.

type annotations

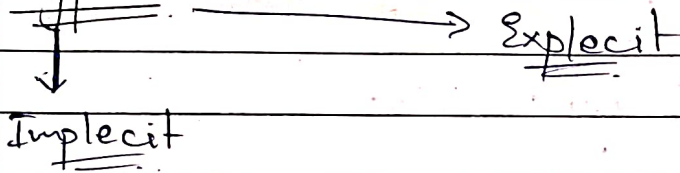
let myNumberExOne: number = 42;
let myStringExOne: string = "Hello";

Working of TS.

This is missing in JS.



Types



Implicitly defining type.

(Inferencing)

let drink = "chai";

drink = 4. (Type Error)

This will throw an error.

as it implicitly declared

drink as string.

Explicitly defining type.

(Type annotations)

let drink : string

drink = "chai"

drink = 4.

let drink : string = "chai"

or drink = 4

→ Type Error ←

Union C to give 2 acceptable datatypes

let subs : number | string = '1M'

↑ It can take number as well as string.

~~let~~

(Custom type annotations)

let apiReqStatus: 'pending' | 'success' | 'error';

apiReqStatus = 'error'; // fine.

apiReqStatus = 'abc'; // TypeError.

type Guards (a good coding practice)

Unknown type.

- relatively safer than any.
- supports type narrowing.
- Example of type narrowing using 'unknown' type :-
function processValue (value : unknown) {
 if (typeof value === "string") {

 // Inside this block 'value' is safely
 // narrowed to type 'string'.

} else if (typeof value === "number") {

 // here 'value' is safely narrowed to number
 } else {

 // handle other types or throw an error
 }

truthiness

```
function serveAPI(msg?: string) {  
    if (msg) {  
        // code  
    } else {  
        // default code or error  
    }  
}
```

at the time of calling this function:-

```
abc("something") // if (msg) {}  
abc()             // else {}  
abc(2)            // typeError
```

Custom types

- they are made using 'type' keyword.
- a validation function should be defined for the type.

Custom type.

```
{  
    type ChaiOrder = {  
        type: string  
        sugar: number  
    }  
}
```

Validation fn.

```
{  
    function isChaiOrder(obj: any): obj is ChaiOrder {  
        return (typeof obj === "object" && obj !== null &&  
            typeof obj.type === "string" && typeof obj.sugar  
                === "number")  
    }  
}
```

Using this custom type.

```
function serveOrder(item: ChairOrder | string) {  
  if (isChairOrder(item)) {  
    return 'serving ${item.type} etc'  
  }  
  return `its string`  
}
```

type assertion.

forceful type
assertion.

let response: any = "42"

let numeric-length: number = (response as string).length

Interfaces.

used to define the shape or structure of an object, class, or function.
Syntax.

interface CupSize {

Size: "small" | "large"

}

Intersection

```
type BaseChai = {teaLeaves: number}
type Extra = {masala: number}
```

```
type MasalaChai = BaseChai & Extra
```

Intersection.

```
const cup: MasalaChai = {
```

both are mandatory. {

```
  teaLeaves: 2,
  masala: 2
}
```

optional values.

```
type user = {
  username: string;
  bio?: string;
}
```

↓
now bio is optional.

Read-only values

```
type config = {
```

```
  readonly appName: string;
  version: number;
}
```

can't change value once defined.